

Docker Assignment

This project is a web application developed using Node.js (Express) as the frontend and Flask (Python) as the backend. The main objective is to demonstrate communication between two different technologies through HTTP requests.

The frontend provides a user interface where users can enter their details, such as name and email. When the form is submitted, the frontend sends the data to the Flask backend using an API request. The backend processes the received data and returns a response in JSON format.

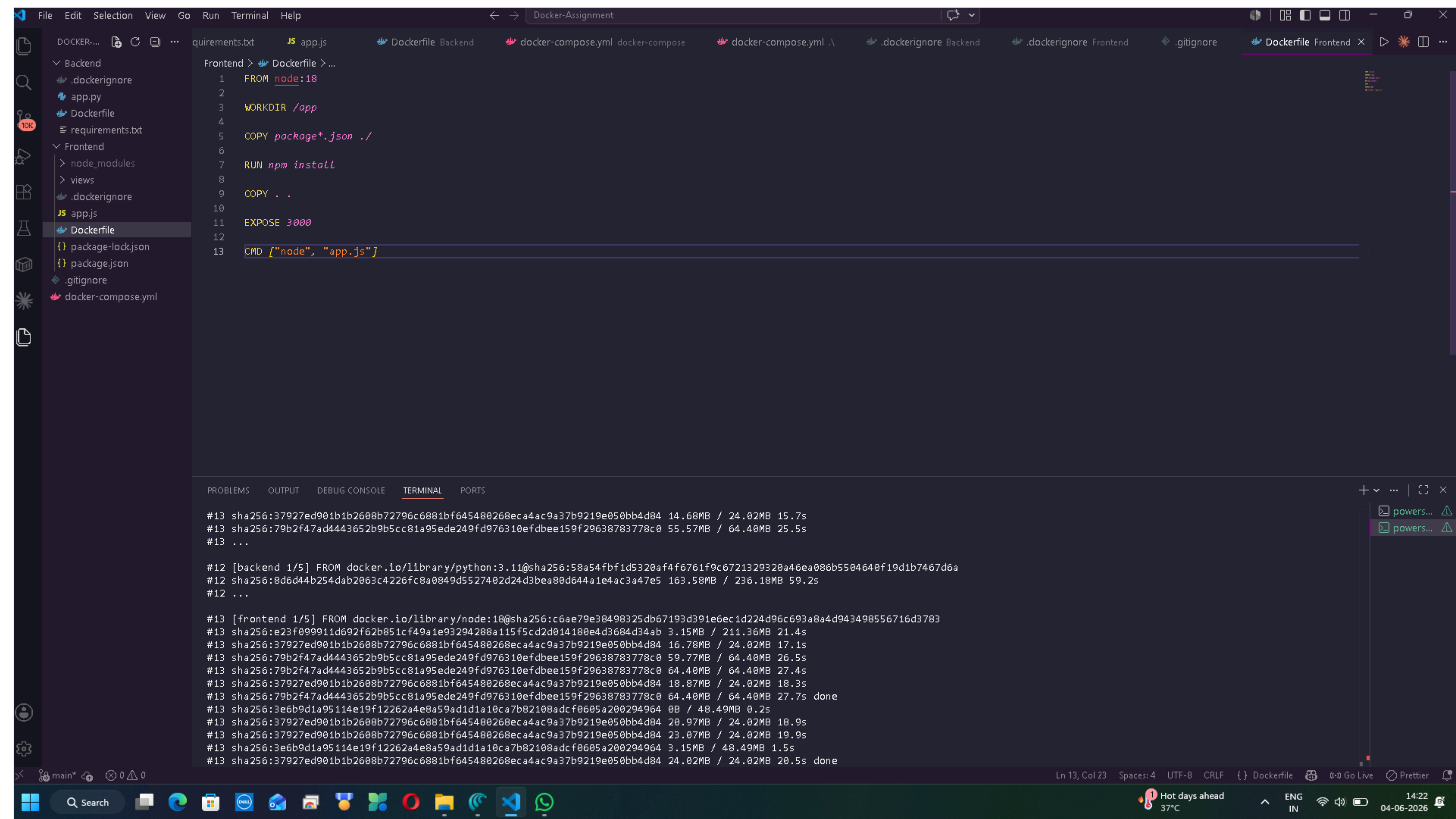
To simplify deployment and ensure consistency across different environments, both the frontend and backend applications are containerized using Docker. Docker Compose is used to manage and run multiple containers together on the same network.

Key Features

User-friendly form interface
Frontend developed using Node.js and Express
Backend developed using Flask
Data exchange through REST API
Docker containerization
Docker Compose for multi-container management
Version control using Git and GitHub

Project Outcome

This project helps in understanding:
Frontend and backend integration
API communication
Docker image creation
Docker Compose configuration
GitHub project management



Technologies Used

Frontend

Node.js

Express.js

EJS

Axios

Backend

Python

Flask

Containerization

Docker

Docker Compose

Version Control

Git

GitHub

Folder Structure

Docker-Assignment

└── Backend

└── app.py

└── requirements.txt

└── Dockerfile

|

└── Frontend

└── app.js

└── package.json

└── views

└── index.ejs

└── Dockerfile

|

└── docker-compose.yml

└── .gitignore

└── README.md

Frontend Implementation

Features

Form Creation

User Input Collection

HTTP POST Request

Response Display

Technologies

Express.js

Axios

EJS Template Engine

Backend Implementation

Flask API

Receives Name and Email

Processes Data

Returns JSON Response

Sample Response

JSON

```
{  
  "name": "Shakti Kumar Behera",  
  "email": "example@gmail.com",  
  "message": "Data Received"  
}
```


Docker Configuration

Backend Dockerfile

Python Base Image

Install Flask

Expose Port 5000

Frontend Dockerfile

Node Base Image

Install Dependencies

Expose Port 3000

Docker Compose

Purpose

Run Multiple Containers

Connect Frontend and Backend

Create Common Network

Services

Backend → Port 5000

Frontend → Port 3000

Conclusion

Successfully created Node.js frontend.

Successfully created Flask backend.

Implemented frontend-backend communication.

Created Dockerfiles.

Configured Docker Compose.

Uploaded project to GitHub.



**THANK
YOU!**